



MIAGE- L3

Systèmes Informatiques

Système d'exploitation

Correction TD 1

BAKAYOKO Ibrahima

Enseignant-chercheur en informatique

UFR Mathématiques & Informatique

Bakayoko.ibrahima1@ufhb.edu.ci

Open Our Mind for a free world

EXERCICE 1

Dans un système d'exploitation un processus ne peut être créé que par un autre processus à l'aide de l'appel système **fork()**. Expliquez comment le premier processus est créé

Le premier processus, appelé **init**, est créé à partir de zéro par le noyau du système d'exploitation lors du processus de démarrage. Le processus **init** démarre d'autres processus dans le système, lesquels peuvent à leur tour en créer davantage, et ainsi de suite.

Le premier processus dans un système d'exploitation est spécialement créé lors du démarrage du système. Contrairement aux processus ultérieurs, il n'est pas créé à l'aide de l'appel système `fork()`. Au lieu de cela, sa création est une partie intégrante du processus de démarrage du système.

Voici comment le premier processus est généralement créé :

Amorçage du Système : Lorsque vous démarrez ou allumez votre ordinateur, le matériel (par exemple, la carte mère) exécute une séquence d'amorçage, également appelée amorçage du système.

Cette séquence charge le noyau du système d'exploitation en mémoire depuis une source de stockage, telle qu'un disque dur ou une mémoire flash.

Initialisation du Noyau : Une fois que le noyau du système d'exploitation est en mémoire, il commence son processus d'initialisation. Le noyau crée le premier processus, souvent appelé **init** (dans les systèmes basés sur Unix/Linux) ou **systemd** (dans certaines distributions Linux modernes).

Ce premier processus est le point de départ de tous les autres processus dans le système.

Exécution du Premier Processus : Le premier processus commence à s'exécuter et est responsable du lancement et de la gestion des autres processus du système.

Lancement d'Applications et de Services : Le premier processus est responsable du démarrage des services système essentiels, tels que le gestionnaire de fenêtres, les services réseau, etc. Il peut également lancer des applications utilisateur selon les besoins.

Création de Processus Subséquents : Une fois que le premier processus est en cours d'exécution, il peut créer des processus subséquents au besoin, généralement en utilisant l'appel système `fork()` ou d'autres mécanismes de création de processus.

En résumé, le premier processus dans un système d'exploitation est spécialement créé par le noyau lors du démarrage du système. Il sert de parent à tous les autres processus et est responsable du lancement et de la gestion des processus système et des applications. Les processus ultérieurs peuvent être créés par le premier processus ou par d'autres processus existants en utilisant des appels système tels que `fork()`.

EXERCICE 2

Quelles sont les similitudes entre un appel système, un changement de contexte et une interruption matérielle ?
Quelles sont les différences ?

Similitudes : dans les deux cas le processus en cours est interrompu contre son gré. Le système d'exploitation reprend la main et sauvegarde l'état du processus avant de gérer le changement de contexte, l'appel système, ou l'entrée-sortie correspondant à l'interruption matérielle.

Différences : un appel système est initié par une instruction logicielle du processus en cours, alors qu'un changement de contexte est initié par l'expiration d'un timer matériel et une interruption matérielle est initiée par la fin d'une entrée/sortie. Dans le cas d'un appel système ou d'un changement de contexte, le système d'exploitation redonne la main à un autre processus, alors que dans le cas d'une interruption matérielle il rend souvent la main au même processus qui s'était fait interrompre.

Les appels système, les changements de contexte et les interruptions matérielles sont tous des mécanismes essentiels dans le fonctionnement des systèmes d'exploitation, mais ils servent des objectifs différents et ont des caractéristiques distinctes.

Similitudes entre ces concepts :

1. Transition de Mode : Les trois concepts impliquent une transition entre différents modes d'exécution du processeur. Les systèmes d'exploitation gèrent généralement deux modes, le mode utilisateur et le mode superviseur (ou noyau). Les appels système, les changements de contexte et les interruptions matérielle s'accompagnent souvent d'un changement de mode.

2. Interaction avec le Noyau : Tous ces mécanismes interagissent directement avec le noyau du système d'exploitation. Les appels système sollicitent des services du noyau, les changements de contexte sont déclenchés par le noyau pour gérer l'exécution de différents processus, et les interruptions matérielle sont des signaux envoyés par le matériel au noyau.

Différences entre ces concepts :

1. Objectifs :

Les appels système sont utilisés par les processus en mode utilisateur pour demander des services ou effectuer des opérations spécifiques, telles que l'ouverture d'un fichier. Ils sont initiés volontairement par les programmes utilisateur.

Les changements de contexte sont effectués par le noyau pour passer d'un processus à un autre, permettant l'exécution multitâche. Ils visent à maintenir l'ordonnancement des processus et à partager le temps CPU entre eux. Les interruptions matérielle sont générées par le matériel en réponse à des événements spécifiques, tels que les demandes d'E/S ou les exceptions matérielles. Elles interrompent l'exécution normale du processeur pour gérer ces événements.

2. Déclenchement :

Les appels système sont déclenchés par des instructions spécifiques exécutées par un processus utilisateur. Ils sont initiés par le programme en cours. Les changements de contexte sont initiés par le noyau pour basculer d'un processus à un autre en fonction d'une politique d'ordonnancement.

Ils se produisent de manière prévue par le système. Les interruptions matérielle sont déclenchées par des événements matériels imprévisibles, tels que des erreurs matérielles, des interruptions d'E/S ou des minuteriers. Elles sont déclenchées par des composants matériels.

3. Réponse :

Les appels système et les changements de contexte sont généralement des opérations synchrones, ce qui signifie que le programme sait quand ils se produisent et peut attendre leur achèvement.

Les interruptions matérielle sont asynchrones. Elles surviennent à des moments imprévisibles et peuvent interrompre l'exécution en cours. Le noyau doit réagir rapidement pour les gérer.

2. Exécution de Code :

Les appels système et les changements de contexte impliquent généralement une exécution continue de code, soit dans l'espace utilisateur (appels système) ou dans le noyau (changement de contexte).

Les interruptions matérielle entraînent une interruption immédiate de l'exécution en cours pour gérer l'interruption, puis la reprise de l'exécution normale.

En résumé, bien que les appels système, les changements de contexte et les interruptions matérielle soient tous liés au fonctionnement des systèmes d'exploitation, ils diffèrent par leurs objectifs, leur déclenchement, leur réponse et leur mode d'exécution. Chacun d'eux remplit un rôle spécifique dans la gestion et le contrôle des ressources et des processus d'un système informatique.

EXERCICE 3

On veut concevoir une nouvelle discipline d'ordonnancement de processus pour un nouveau système d'exploitation. Dans ce nouveau système les processus n'ont pas de niveaux de priorité mais ils ont un attribut explicite qui les classe soit en catégorie "I/O-bound" soit en "CPU-bound".

Une discipline d'ordonnancement doit essentiellement choisir quel processus doit s'exécuter en premier dans les situations suivantes :

- Deux processus "I/O-bound" sont en état WAITING
- Deux processus "CPU-bound" sont en état WAITING

Un processus "I/O-bound" et un processus "CPU-bound" sont en état WAITING. Proposez une discipline d'ordonnancement simple pour chacun des trois cas, et expliquez pourquoi cette discipline est raisonnable.

Une politique simple est la suivante :

les processus I/O-bound utilisent intensivement des ressources différentes, nous pouvons donc les planifier indépendamment les uns des autres. Nous utilisons une politique de type "round-robin" (tourniquet) parmi tous les processus CPU-bound, et tous les processus I/O-bound .

Si un processus CPU-bound et un processus I/O-bound sont tous deux dans l'état d'ATTENTE, nous choisissons toujours celui lié à l'E/S (I/O-bound) : nous nous attendons à ce qu'il utilise le CPU pendant une courte période avant d'être bloqué, tandis que le processus lié au CPU (CPU-bound) pourrait occuper le CPU pendant une durée beaucoup plus longue.

Il existe bien sûr de nombreuses autres politiques possibles. Les politiques acceptables sont celles qui tiennent compte de la distinction entre les processus liés au CPU (CPU-bound) et ceux liés à l'E/S (I/O-bound), et sont accompagnées d'une explication plausible.

Pour concevoir une discipline d'ordonnancement simple pour un système d'exploitation où les processus sont classés comme "I/O-bound" ou "CPU-bound" sans niveaux de priorité, nous pouvons utiliser une approche qui tire parti des caractéristiques intrinsèques de ces deux types de processus.

Proposition de discipline d'ordonnancement pour chacun des trois cas mentionnés :

Cas 1 : Deux processus "I/O-bound" en état WAITING

Ici, où deux processus "I/O-bound" attendent des opérations d'E/S, nous pouvons adopter une politique simple de type "round-robin". Cela signifie que nous alternerons l'exécution entre les deux processus "I/O-bound" en attente. L'idée est de donner à chaque processus une opportunité égale d'accéder aux ressources d'E/S.

Cas 2 : Deux processus "CPU-bound" en état WAITING :

Lorsque deux processus "CPU-bound" sont en état WAITING, cela signifie qu'ils attendent des ressources de calcul, comme le CPU lui-même. Dans ce cas, nous pouvons également utiliser une politique de type "round-robin" pour ces processus. Le but est d'assurer une répartition équitable du temps CPU entre eux, ce qui permet à chaque processus "CPU-bound" d'obtenir une part égale du CPU lorsqu'il est disponible.

Cas 3 : Un processus "I/O-bound" et un processus "CPU-bound" en état WAITING :

Lorsque nous avons un processus "I/O-bound" et un processus "CPU-bound" en attente, nous pouvons prioriser le processus "I/O-bound". La raison en est que les processus "I/O-bound" ont tendance à libérer rapidement le CPU une fois qu'ils ont terminé leurs opérations d'E/S, tandis que les processus "CPU-bound" peuvent monopoliser le CPU pendant une période prolongée.

En donnant la priorité au processus "I/O-bound", nous pouvons minimiser les temps d'attente pour les opérations d'E/S, ce qui peut améliorer la réactivité globale du système.

En résumé, les disciplines d'ordonnancement proposées sont simples et logiques en fonction des caractéristiques des processus "I/O-bound" et "CPU-bound". Elles visent à équilibrer l'utilisation des ressources du système tout en tenant compte de la nature des opérations que chaque type de processus effectue.

Cependant, il est important de noter que ces politiques sont basées sur des hypothèses générales et que la conception précise de l'ordonnanceur peut dépendre de plusieurs autres facteurs, tels que la charge de travail du système et les objectifs de performance spécifiques.

EXERCICE 4

Le programme ci-dessous comprend une "race condition" si on l'exécute plusieurs fois simultanément. Montrez un exemple d'exécution qui illustre cette race condition.

```
#define TAILLE_CHAINE 1024
char * chaine ; /* variable en memoire partagée */
void ajouter_mot (char *mot) {
    int longueur; /* variable locale */
    /* mesurer la longueur actuelle de la chaine */
    longueur = strlen(chaine) ;
    /* Ecrire mot dans la chaine a partir du caractere chaine [longueur] */
    strncpy (chaine+longueur, mot, TAILLE_CHAINE – longueur);
}
```

Il suffit qu'un premier processus se fasse interrompre avant la ligne `strncpy` et qu'un deuxième processus exécute l'ensemble de la fonction avant que le 1er ne finisse la fonction.

Le code semble manipuler une variable partagée chaîne en mémoire partagée, ce qui peut potentiellement conduire à des conditions de concurrence (race conditions). Si le code est exécuté simultanément par plusieurs **threads** ou **processus**. Une race condition se produit lorsque deux ou plusieurs threads ou processus tentent de modifier la même variable en même temps, ce qui peut entraîner des résultats inattendus ou incorrects.

Dans ce code, la race condition potentielle réside dans la manière dont la longueur de la chaîne est mesurée (**longueur = strlen(chaine);**) et comment elle est modifiée par la suite (**strncpy(chaine + longueur, mot, TAILLE_CHAINE - longueur);**). Si deux exécutions concurrentes du code tentent de modifier la chaîne en même temps, elles peuvent lire et écrire simultanément dans la même région mémoire, provoquant ainsi une race condition.

Exemple simplifié d'exécution concurrente qui illustre cette race condition :

Supposons que deux processus, A et B, exécutent le code suivant en même temps :

Thread A :

```
longueur = strlen(chaine);
```

```
// longueur vaut, par exemple, 5
```

```
// En même temps, le thread B exécute le même code
```

```
strncpy(chaine + longueur, "XYZ", TAILLE_CHAINE - longueur);
```

```
// Dans le contexte de Thread A, la longueur est toujours 5
```

Thread B :

```
longueur = strlen(chaine);
```

```
// longueur vaut, par exemple, 5
```

```
// En même temps, le thread A exécute le même code
```

```
strncpy(chaine + longueur, "ABC", TAILLE_CHAINE - longueur);
```

```
// Dans le contexte de Thread B, la longueur est toujours 5
```

Dans cet exemple, les deux threads A et B mesurent la longueur de la chaîne simultanément et obtiennent la même valeur (5). Ensuite, ils tentent tous les deux d'ajouter leur propre mot ("XYZ" pour A et "ABC" pour B) à la même position (chaîne + 5). En conséquence, les modifications concurrentes peuvent écraser les données les unes des autres, entraînant un état incorrect de la variable chaîne.

Pour éviter cette race condition, vous devrez synchroniser l'accès à la variable partagée chaîne en utilisant des mécanismes de synchronisation tels que les verrous (mutex) ou d'autres techniques de synchronisation appropriées pour votre environnement d'exécution (comme les sémaphores ou les opérations atomiques). Cela permettra de garantir que seul un thread ou un processus accède à la variable chaîne à la fois, évitant ainsi les conflits et les résultats incorrects.

Un sémaphore est un mécanisme de synchronisation utilisé dans les systèmes d'exploitation et la programmation concurrente pour contrôler l'accès concurrent à des ressources partagées ou pour coordonner l'exécution de plusieurs threads ou processus. Les sémaphores sont généralement des entiers non négatifs qui sont utilisés pour effectuer des opérations de verrouillage et de déverrouillage.

EXERCICE 5

Pour tenter de résoudre cette race condition, un programmeur propose le programme suivant :

```
#define TAILLE_CHAINE 1024
char *chaine ; /* variable en mémoire partagée */
int verrou ; /* variable en mémoire partagée */
void rajouter_mot (char *mot) {
    int longueur ; /* variable locale */
    while (verrou != 0) { } /* attendre que le verrou se libère */
    verrou = 1 ; /* verrouiller le verrou */
    longueur = strlen (chaine) ; /* la longueur de la chaîne */
    strncpy (chaine+longueur , mot , TAILLE_CHAINE - longueur ) ;
    verrou = 0 ; /* deverrouiller le verrou */
    return 0 ;
}
```

Est-ce que la "race condition" est toujours possible ? Expliquez votre réponse.

Oui, la "race condition" est toujours présente. Il suffit qu'un premier processus se fasse interrompre avant la ligne "verrou=1;".

Le programme utilise une variable verrou pour tenter de résoudre la race condition en ajoutant une section critique où les threads ou les processus concurrents doivent attendre que le verrou soit libéré avant d'accéder à la variable partagée chaîne. Cette approche est appelée "verrouillage manuel" ou "verrouillage binaire". Cependant, il y a encore un problème potentiel dans ce code qui peut entraîner des problèmes de concurrence.

Le code semble tenter de synchroniser l'accès à la variable chaîne en utilisant la variable verrou.

Voici comment cela fonctionne :

Lorsqu'un thread ou un processus souhaite accéder à la variable chaîne, il vérifie d'abord la valeur de verrou. S'il est différent de zéro (ce qui signifie que le verrou est actuellement détenu par un autre thread ou processus), il reste en boucle en attendant que le verrou soit libéré.

Une fois que le thread ou le processus a réussi à vérifier que le verrou est égal à zéro, il verrouille le verrou en le mettant à 1, ce qui empêche les autres threads ou processus d'accéder à la section critique.

Ensuite, le thread ou le processus accède à la variable chaîne, effectue les opérations nécessaires, puis libère le verrou en le remettant à zéro.

Bien que cette approche puisse sembler résoudre le problème de la race condition en garantissant que seul un thread ou un processus à la fois puisse accéder à la variable chaîne, elle présente toujours un problème potentiel : le verrouillage manuel. Le principal problème réside dans la boucle d'attente `while (verrou != 0) {}`.

Cette boucle d'attente active (également appelée "polling") peut entraîner une utilisation intensive du processeur, car un thread en attente du verrou vérifie constamment sa valeur. Cela peut gaspiller des ressources du processeur et réduire l'efficacité globale du système.

De plus, il existe un risque de blocage (deadlock) si un thread ou un processus détient le verrou de manière permanente (par exemple, en cas de panne ou d'erreur). Dans ce cas, tous les autres threads ou processus qui tentent d'acquérir le verrou resteront bloqués indéfiniment, ce qui entraîne un blocage du programme.

Pour éviter ces problèmes, il est généralement recommandé d'utiliser des mécanismes de synchronisation plus sophistiqués, tels que les mutex, qui sont spécialement conçus pour éviter les problèmes de blocage et minimiser l'utilisation intensive du processeur. Les mutex sont généralement fournis par les bibliothèques de threads ou les systèmes d'exploitation et offrent une meilleure gestion des ressources et de la concurrence.